INTERVIEW PREPARATION

Mobile Test Automation — Interview Guide

Concepts behind the eGenome project, likely questions, STAR stories and key notes · 30 Oct 2026 · Nishant

Interview deep dive: the concepts behind the project

This section turns the project into interview knowledge. Each topic gives the concept, how this project applies it, and the question it usually comes up as.

How Appium works

- **Client–server over the W3C WebDriver protocol.** Your test (the client) sends HTTP/JSON requests such as "find element" or "click" to the Appium server. Appium forwards them to a platform driver that acts on the device.
- **Sessions and capabilities.** Each run starts a session described by capabilities: `platformName`, `appium:automationName` (UiAutomator2), `appium:appPackage` / `appActivity`, `appium:udid`, `appium:noReset`, timeouts. Wrong capabilities are the most common reason a session won't start.
- **Drivers are plug-ins since Appium 2.** You install them separately (`appium driver install uiautomator2`). Android: UiAutomator2 or Espresso. iOS: XCUITest.
- **UiAutomator2 vs Espresso.** UiAutomator2 is black-box: it installs a server APK and a test APK on the device and can touch any app, including system dialogs. Espresso runs inside the app's process, so it is faster and syncs with the UI thread automatically, but it needs the app's APK re-signed and can't leave the app.
- *Typical question:* "Walk me through what happens when you call `element.click()` in Appium." Answer with the chain above: client → HTTP → Appium server → UiAutomator2 server on the device → Android UiAutomator → the app.

Locator strategies

- **Preference order:** accessibility id (`~label`, works on Android and iOS) → resource-id → `-android uiautomator` selectors → XPath as a last resort. XPath is slow on mobile because the whole UI tree is serialised to XML first, and absolute or index-based paths break on any layout change.
- **In this project:** the app is React Native, so most elements have no resource-id. We used accessibility labels with partial matching (`descriptionContains`, `descriptionMatches`).
- **The real fix is in the app.** Ask developers to add `testID` / `accessibilityLabel` to interactive elements. Saying this in an interview shows you work with developers, not around them.
- *Typical question:* "How do you choose a locator, and what do you do when an element has no ID?"

Synchronisation and waits

- **Implicit wait:** one global timeout for every element lookup. Simple, but hides slowness and mixes badly with explicit waits.
- **Explicit wait:** wait for a specific condition (`waitForDisplayed`, `waitForEnabled`, `browser.waitUntil(fn)`). This project uses explicit waits only.
- **Never use fixed sleeps** (`pause(5000)`): too short on a slow device, wasted time on a fast one.
- **Idle waits:** UiAutomator2 waits for the UI to be idle before each command. On a screen with a constant animation this made every command take 10 seconds; lowering `waitForIdleTimeout` fixed it.
- *Typical question:* "What's the difference between implicit and explicit waits, and why shouldn't you mix them?"

Page Object Model and other patterns

- **Page Object Model (POM):** one class per screen holds its locators and actions; tests read like user stories. A UI change means editing one class.
- **Components:** shared parts (bottom tab bar, permission dialogs) are their own objects and reused across pages.

- **Base class:** common behaviour (tap, type, scroll, wait, verify layout) lives once in `BasePage`.
- **Where assertions go:** keep them in tests; page objects return data or state. That keeps pages reusable for both positive and negative tests.
- **Alternatives to know by name:** the Screenplay pattern (actors, tasks, questions) and BDD with Cucumber (Given/When/Then feature files on top of page objects).
- *Typical question:* "Why use Page Objects? What problems do they not solve?" (They don't fix bad locators or flaky timing.)

Test design and suite strategy

- **Test pyramid:** many unit tests, fewer API tests, fewest UI tests. UI tests are slow and fragile, so each one must cover a valuable user journey.
- **Smoke vs sanity vs regression:** smoke = "is the build usable at all" (our 5-minute suite); sanity = a quick check of one changed area; regression = everything that used to work still works.
- **Priorities:** P0 = blocks the core journey (login, view report); P1 = important but has a workaround; P2 = edge cases. Run P0 on every build.
- **Techniques:** equivalence partitioning and boundary values (empty, short, letters in the phone field), negative tests (wrong OTP), state transitions (sign out, restart, background).
- **Independence:** every test sets up its own state and cleans up after itself, so tests can run alone, in any order, or in parallel.
- *Typical question:* "How do you decide what to automate?" Answer: repeatable, stable, high-risk, high-value flows; leave exploratory, one-off and rapidly changing UI to manual testing.

Flaky tests

- **Common causes:** timing (no proper wait), shared state between tests, test order dependence, animations, network or backend instability, test data that changes.
- **Fixes, in order:** find the root cause (screenshots, logs, page source) → replace sleeps with explicit waits → isolate state → stabilise data → only then add retries (`specFileRetries`), and track which tests needed them.
- **Measure it:** the dashboard's test-health table flags a test as flaky when it both passed and failed in recent runs.
- *Typical question:* "A test passes locally but fails in CI. How do you debug it?"

Mobile-specific testing

- **Emulators vs real devices:** emulators are fast and repeatable for CI; real devices catch OEM differences (Samsung, Xiaomi, OnePlus dialogs and settings), performance, sensors and real networks. Use both, or a device cloud.
- **Fragmentation:** screen sizes, Android versions and OEM skins. Proportional layout checks and relation rules handle different screen sizes better than pixel positions.
- **App lifecycle:** install, launch, background, kill, restore, clear data (`mobile: clearApp`, `activateApp`, `terminateApp`).
- **Permissions and system dialogs:** auto-grant via capability, or handle pop-ups explicitly; test the "denied" path too.
- **Gestures:** scroll, swipe, long-press via `mobile:` commands or W3C Actions.
- **OTP / 2FA:** use a staging test number with a fixed code or a backend bypass; never automate real SMS.
- **Network:** toggle Wi-Fi/data to test offline behaviour and recovery.

Visual and layout testing

- **Pixel comparison** (Applitools, Percy, wdio-image-comparison): catches any visual change, but breaks on fonts, animations and devices; needs tolerance and masking.
- **Structural comparison** (this project): compares the element tree — text, presence, proportional position, alignment, order. Robust across devices, but blind to colours and images.

- Knowing the trade-off, and that the two complement each other, is what interviewers look for.

Reporting and CI/CD

- **Reports:** console reporter for developers, HTML reports (Allure or a custom dashboard) for the team, with a screenshot and logs on every failure.
- **CI:** run smoke on every pull request and regression nightly. GitHub Actions can boot an Android emulator; device clouds (BrowserStack, Sauce Labs, Firebase Test Lab) give real devices at scale.
- **Parallel runs:** one Appium session per device; `maxInstances` and several devices (or cloud devices) shorten the run.
- **Gating:** the test command's exit code fails the pipeline, so a red smoke run blocks the merge or release.

JavaScript you must be fluent in

- `async` / `await` and Promises — every device command is asynchronous; a missing `await` is the classic bug.
- ES modules (`import` / `export`), classes and getters, arrow functions, destructuring, template literals.
- Array methods: `map`, `filter`, `find`, `reduce`, `some`, `every`.
- Closures and higher-order functions (the `S` selector helpers return functions).
- `Proxy`, used here to record every page-object call as a dashboard step — a good "advanced JavaScript" talking point.

Interview pointers and notes

Your 60-second project pitch

> *I built an end-to-end test framework for a React Native health app on Android, using WebdriverIO, Appium and UiAutomator2 with the Page Object Model. It has a 5-minute smoke suite that runs on every build and a 60-test regression suite covering all P0 and P1 flows, including OTP login. On top of that I added structural layout checks that compare each screen's elements by proportion, so they work across devices, and a custom dashboard that records every step with a screenshot. The hardest problems were flaky timing on animated screens and selectors that broke on the device — I fixed both by reading the raw page source and the Appium settings rather than adding retries.*

Numbers to quote

Item	Value
Automated tests	84 (smoke 8, regression 60, layout 16) + 25 planned
Screens with page objects	26
Captured locators	214
Screens with layout specs	16
Smoke run time	about 5 minutes
Layout checks per smoke run	about 3,700

Questions you should be ready for

Question	Short answer
Why WebdriverIO and not Java + TestNG?	Same WebDriver protocol underneath; JavaScript matched the React Native team's stack, and WebdriverIO has Appium built in
How do you test OTP login?	Staging test number with a fixed OTP; never real SMS. Negative tests for wrong OTP and the resend timer
How do you keep tests independent?	Each spec reaches a known state (`ensureLoggedIn` , clear app data); tests restore any data they change

Question	Short answer
How do you handle a pop-up that sometimes appears?	A helper that checks for it with a short timeout and dismisses it, called at known points — not a blind try/catch everywhere
How do you run on many devices?	Device ID from config; one session per device; parallel runs with `maxInstances` or a device cloud
What would you improve next?	CI pipeline on every pull request, `testID`s from developers, API calls for test setup, a per-device layout baseline
How do you report a bug from a failed test?	Dashboard step + screenshot + logcat + steps to reproduce + expected vs actual

Problem-solving stories (STAR format)

1. **Slow screen.** *Situation:* on Manage Addresses every command took 40–90 s and the smoke test timed out. *Task:* find out why only that screen. *Action:* compared step timings and saw an animated card; UiAutomator2 waits for an idle UI before each command. Lowered `waitForIdleTimeout` to 100 ms. *Result:* the screen went from 91 s to 2 s.
2. **Selector that never matched.** *Situation:* "2 reports found" was on screen but the test couldn't find it. *Action:* noticed only regex selectors with `\d` failed; the backslash was being doubled on the way to the device. Rewrote patterns without backslashes (`[0-9]`) in one central helper. *Result:* fixed every pattern selector in the suite at once.
3. **One failure failing everything.** *Situation:* a layout failure left a screen open, so every later test failed too. *Action:* made tab navigation recover by pressing Back until the tab bar is visible. *Result:* failures stay isolated to the test that caused them.

Notes and pitfalls to remember

- **Always `await`** every WebdriverIO call; a missing `await` gives confusing, intermittent failures.
- **Read the first failure,** not the last screenshot — later failures are often knock-on effects.
- **Dump the page source** when an element can't be found; it shows what Android actually reports.
- **Never commit secrets:** `.env` stays out of git; `.env.example` documents the variables.
- **Retries hide problems.** Use them only after fixing the root cause, and track which tests needed them.
- **Real devices need one-time human steps** (USB debugging, install prompts, OEM settings) — document them.
- **Know every part well enough to explain it in your own words.** Interviewers will ask why you made each choice, not just what the code does.
- **Glossary to have ready:** capability, session, driver, locator, page source, explicit wait, page object, smoke, regression, flaky test, test pyramid, CI pipeline.